A type trait to detect reference binding to temporary

Document #:	P2255R2
Date:	2021-10-13
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Tim Song <t.canens.cpp@gmail.com>

Contents

§ 1 Abstract

This paper proposes adding two new type traits with compiler support to detect when the initialization of a reference would bind it to a lifetime-extended temporary, and changing several standard library components to make such binding ill-formed when it would inevitably produce a dangling reference. This would resolve [LWG2813].

§ 2 Revision history

— R2: Approved by 2021-05-12 EWG telecon. Added a feature test macro. Also modify `make_from_tuple` and add a note to the piecewise constructor of `pair`. Rebase wording to post-2021-10 virtual plenary working draft.

— R1: Define the affected constructor of `tuple` and `pair` as deleted instead of removing them from overload resolution.

§ 3 In brief

Before	After

`std::tuple<const std::string&> x("hello"); // dangling`
`std::function<const std::string&()> f = [] { return ""; }; // OK`
	`std::tuple<const std::string&> x("hello"); // ill-formed`
	`std::function<const std::string&()> f = [] { return ""; }; // ill-formed`

Before	After
`f();` `// dangling`	

§ 4 Motivation

Generic libraries, including various parts of the standard library, need to initialize an entity of some user-provided type `T` from an expression of a potentially different type. When `T` is a reference type, this can easily create dangling references. This occurs, for instance, when a `std::tuple<const T&>` is initialized from something convertible to `T`:

```
std::tuple<const std::string&> t("meow");
```

This construction *always* creates a dangling reference, because the `std::string` temporary is created *inside* the selected constructor of `tuple` (`template<class... UTypes> tuple(UTypes&&...)`), and not outside it. Thus, unlike `string_view`'s implicit conversion from rvalue strings, under no circumstances can this construction be correct.

Similarly, a `std::function<const string&()>` currently accepts any callable whose invocation produces something convertible to `const string&`. However, if the invocation produces a `std::string` or a `const char*`, the returned reference would be bound to a temporary and dangle.

Moreover, in both of the above cases, the problematic reference binding occurs inside the standard library's code, and some implementations are known to suppress warnings in such contexts.

§ 4.1 Related work

[P0932R1] proposes modifying the constraints on `std::function` to prevent such creation of dangling references. However, the proposed modification is incorrect (it has both false positives and false negatives), and correctly detecting all cases in which dangling references will be created without false positives is likely impossible or at least heroically difficult without compiler assistance, due to the existence of user-defined conversions.

[CWG1696] changed the core language rules so that initialization of a reference data member in a *mem-initializer* is ill-formed if the initialization would bind it to a temporary expression, which is exactly the condition these traits seek to detect. However, the ill-formedness occurs outside a SFINAE context, so it is not usable in constraints, nor suitable as a `static_assert` condition. Moreover, this requires having a class with a data member of reference type, which may not be suitable for user-defined types that want to represent references differently (to facilitate rebinding, for instance).

§ 5 Design decisions

§ 5.1 Alternative approaches

Similar to [CWG1696], we can make returning a reference from a function ill-formed if it would be bound to a temporary. Just like [CWG1696], this cannot be used as the basis of a constraint or as a `static_assert` condition. Additionally, such a change requires library wording to react, as `is_convertible` is currently defined in terms of such a return statement. While such a language change may be desirable, it is neither necessary nor sufficient to accomplish the goals of this paper. It can be proposed separately if desired.

During a previous EWG telecon discussion, some have suggested inventing some sort of new initialization rules, perhaps with new keywords like `direct_cast`. The author of this paper is unwilling to spare a kidney for any new keyword in this area, and such a construct can easily be implemented in the library if the traits are available. Moreover, changing initialization rules is a risky endeavor; such changes frequently come with unintended consequences (for

recent examples, see [gcc-pr95153] and [LWG3440]). It's not at all clear that the marginal benefit from such changes (relative to the trait-based approach) justifies the risk.

§ 5.2 Two type traits

This paper proposes two traits, `reference_constructs_from_temporary` and `reference_converts_from_temporary`, to cover both (non-list) direct-initialization and copy-initialization. The former is useful in classes like `std::tuple` and `std::pair` where `explicit` constructors and conversion functions may be used; the latter is useful for *`INVOKE<R>`* (e.g., `std::function`) where only implicit conversions are considered.

As is customary in the library traits, "construct" is used to denote direct-initialization and "convert" is used to denote copy-initialization.

§ 5.3 Treat prvalues as distinct from xvalues

Unlike most library type traits, this paper proposes that the traits handle prvalues and xvalues differently: `reference_converts_from_temporary<int&&, int>` is `true`, while `reference_converts_from_temporary<int&&, int&&>` is `false`. This is useful for *`INVOKE<R>`*; binding an rvalue reference to the result of an xvalue-returning function is not incorrect (as long as the function does not return a dangling reference itself), but binding it to a prvalue (or a temporary object materialized therefrom) would be.

§ 5.4 Changing *`INVOKE<R>`* and `is_invocable_r`

Changing the definition of *`INVOKE<R>`* as proposed means that `is_invocable_r` will also change its meaning, and there will be cases where `R v = std::invoke(args...);` is valid but `is_invocable_r_v<R, decltype((args))...>` is `false`:

```
auto f = []{ return "hello"; };

const std::string& v = std::invoke(f);                              // OK
static_assert(is_invocable_r_v<const std::string&, decltype((f))>); // now fails
```

However, we already have the reverse case today (`is_invocable_r_v` is `true` but the declaration isn't valid, which is the case if `R` is *cv* `void`), so generic code already cannot use `is_invocable_r` for this purpose.

More importantly, actual usage of *`INVOKE<R>`* in the standard clearly suggests that changing its definition is the right thing to do. It is currently used in six places:

- `std::function`
- `std::visit<R>`
- `std::bind<R>`
- `std::packaged_task`
- `std::invoke_r`
- `std::move_only_function`

In none of them is producing a temporary-bound reference ever correct. Nor would it be correct for the proposed `std::function_ref` ([P0792R5]).

§ 5.5 `tuple`/`pair` constructors: deletion vs. constraints

The wording in R0 of this paper added constraints to the constructor templates of `tuple` and `pair` to remove them from overload resolution when the initialization would require binding to a materialized temporary. During LEWG mailing list review, it was pointed out that this would cause the construction to fall back to the `tuple(const Types&...)` constructor instead, with the result that a temporary is created *outside* the `tuple` constructor and then bound to the reference.

While there are plausible cases where doing this is valid (for instance, `f(tuple<const string&>("meow"))`, where the temporary string will live until the end of the full-expression), the risk of misuse is great enough that this revision proposes that the constructor be deleted in this scenario instead. Deleting the constructor still allows the condition to be observable to type traits and constraints, and avoids silent fallback to a questionable overload. Advanced users who desire such a binding can still explicitly convert the string themselves, which is what they have to do for correctness today anyway.

§ 6 Implementation experience

Clang has a `__reference_binds_to_temporary` intrinsic that partially implements the direct-initialization variant of the proposed trait: it does not implement the part that involves reference binding to a prvalue of the same or derived type.

```
static_assert(__reference_binds_to_temporary(std::string const &, const char*));
static_assert(not __reference_binds_to_temporary(int&&, int));
static_assert(not __reference_binds_to_temporary(Base const&, Derived));
```

However, that part can be done in the library if required, by checking that

— the destination type `T` is a reference type;

— the source type `U` is not a reference type (i.e., it represents a prvalue);

— `is_convertible_v<remove_cvref_t<U>*, remove_cvref_t<T>*>` is `true`.

§ 7 Wording

This wording is relative to [N4892] after the application of [P2321R2] and [LWG3121].

1. Edit 20.15.3 [meta.type.synop], header `<type_traits>` synopsis, as indicated:

```
namespace std {
  […]
  template<class T> struct has_unique_object_representations;

+  template<class T, class U> struct reference_constructs_from_temporary;
+  template<class T, class U> struct reference_converts_from_temporary;

  […]

  template<class T>
    inline constexpr bool has_unique_object_representations_v
      = has_unique_object_representations<T>::value;

+  template<class T, class U>
+    inline constexpr bool reference_constructs_from_temporary_v
+      = reference_constructs_from_temporary<T, U>::value;
+  template<class T, class U>
+    inline constexpr bool reference_converts_from_temporary_v
+      = reference_converts_from_temporary<T, U>::value;
```

```
  […]
}
```

2. In 20.15.5.4 [meta.unary.prop], add the following after p4:

? For the purpose of defining the templates in this subclause, let *VAL<T>* for some type *T* be an expression defined as follows:

(?.1) — If *T* is a reference or function type, *VAL<T>* is an expression with the same type and value category as `declval<T>()`.

(?.2) — Otherwise, *VAL<T>* is a prvalue that initially has type *T*. [*Note 1:* If *T* is cv-qualified, the cv-qualification is subject to adjustment (7.2.2 [expr.type]). — *end note*]

3. In 20.15.5.4 [meta.unary.prop], Table 49 [tab:meta.unary.prop], add the following:

Template	Condition	Preconditions
`template<class T, class U>` `struct reference_constructs_from_temporary;`	`conjunction_v<is_reference<T>, is_constructible<T, U>>` is `true`, and the initialization `T t(VAL<U>);` binds `t` to a temporary object whose lifetime is extended (6.7.7 [class.temporary]).	`T` and `U` shall be complete types, *cv* `void`, or arrays of unknown bound.
`template<class T, class U>` `struct reference_converts_from_temporary;`	`conjunction_v<is_reference<T>, is_convertible<U, T>>` is `true`, and the initialization `T t =` *VAL*`<U>;` binds `t` to a temporary object whose lifetime is extended (6.7.7 [class.temporary]).	`T` and `U` shall be complete types, *cv* `void`, or arrays of unknown bound.

4. Edit 20.4.2 [pairs.pair] as indicated:

```
template<class U1 = T1, class U2 = T2> constexpr explicit(see below) pair(U1&& x, U2&&
        y);
```

11 *Constraints:*

(11.1) — `is_constructible_v<first_type, U1>` is `true` and

(11.2) — `is_constructible_v<second_type, U2>` is `true`.

12 *Effects:* Initializes `first` with `std::forward<U1>(x)` and `second` with `std::forward<U2>(y)`.

13 *Remarks:* The expression inside `explicit` is equivalent to: `!is_convertible_v<U1, first_type> || !is_convertible_v<U2, second_type>`. This constructor is defined as deleted if `reference_constructs_from_temporary_v<first_type, U1&&>` is `true` or `reference_constructs_from_temporary_v<second_type, U2&&>` is `true`.

```
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1, U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(const pair<U1, U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1, U2>&& p);
template<class U1, class U2> constexpr explicit(see below) pair(const pair<U1, U2>&& p);
```

14 Let *FWD*(u) be static_cast<decltype(u)>(u).

15 *Constraints*:

(15.1) — is_constructible_v<first_type, decltype(get<0>(*FWD*(p)))> is true and

(15.2) — is_constructible_v<second_type, decltype(get<1>(*FWD*(p)))> is true.

16 *Effects:* Initializes first with get<0>(*FWD*(p)) and second with get<1>(*FWD*(p)).

17 *Remarks:* The expression inside explicit is equivalent to: !is_convertible_v<decltype(get<0>(*FWD*(p))), first_type> || !is_convertible_v<decltype(get<1>(*FWD*(p))), second_type>. The constructor is defined as deleted if reference_constructs_from_temporary_v<first_type, decltype(get<0>(*FWD*(p)))> is true or reference_constructs_from_temporary_v<second_type, decltype(get<1>(*FWD*(p)))> is true.

```
template<class... Args1, class... Args2>
  constexpr pair(piecewise_construct_t,
                 tuple<Args1...> first_args, tuple<Args2...> second_args);
```

[Drafting note: No changes are needed here because this is a *Mandates:* and the initialization is ill-formed under [CWG1696]. However, a note is added to make this explicit]

18 *Mandates:*

(18.1) — is_constructible_v<first_type, Args1...> is true and

(18.2) — is_constructible_v<second_type, Args2...> is true.

19 *Effects:* Initializes first with arguments of types Args1... obtained by forwarding the elements of first_args and initializes second with arguments of types Args2... obtained by forwarding the elements of second_args. (Here, forwarding an element x of type U within a tuple object means calling std::forward<U>(x).) This form of construction, whereby constructor arguments for first and second are each provided in a separate tuple object, is called *piecewise construction*. [*Note ?:* If a data member of pair is of reference type and its initialization binds it to a temporary object, the program is ill-formed (11.9.3 [class.base.init]). *— end note*]

5. Edit 20.5.3.1 [tuple.cnstr] as indicated:

```
template<class... UTypes> constexpr explicit(see below) tuple(UTypes&&... u);
```

12 Let *disambiguating-constraint* be:

(12.1) — negation<is_same<remove_cvref_t<U_0>, tuple>> if sizeof...(Types) is 1;

(12.2) — otherwise, bool_constant<!is_same_v<remove_cvref_t<U_0>, allocator_arg_t> || is_same_v<remove_cvref_t<T_0>, allocator_arg_t>> if sizeof...(Types) is 2 or 3;

(12.3) — otherwise, true_type.

13 *Constraints:*

(13.1) — sizeof...(Types) equals sizeof...(UTypes),

(13.2) — sizeof...(Types) ≥ 1, and

(13.3) — `conjunction_v<`*`disambiguating-constraint`*`, is_constructible<Types, UTypes>...>` is `true`.

14 *Effects:* Initializes the elements in the tuple with the corresponding value in `std::forward<UTypes>(u)`.

15 *Remarks:* The expression inside `explicit` is equivalent to: `!conjunction_v<is_convertible<UTypes, Types>...>`. This constructor is defined as deleted if `(reference_constructs_from_temporary_v<Types, UTypes&&> || ...)` is `true`.

[…]

```
template<class... UTypes> constexpr explicit(see below) tuple(tuple<UTypes...>& u);
template<class... UTypes> constexpr explicit(see below) tuple(const tuple<UTypes...>&
        u);
template<class... UTypes> constexpr explicit(see below) tuple(tuple<UTypes...>&& u);
template<class... UTypes> constexpr explicit(see below) tuple(const tuple<UTypes...>&&
        u);
```

19 Let `I` be the pack `0, 1, ..., (sizeof...(Types) - 1)`. Let *`FWD`*`(u)` be `static_cast<decltype(u)>(u)`.

20 *Constraints*:

(20.1) — `sizeof...(Types)` equals `sizeof...(UTypes)`, and

(20.2) — `(is_constructible_v<Types, decltype(get<I>(`*`FWD`*`(u)))> && ...)` is `true`, and

(20.3) — either `sizeof...(Types)` is not `1`, or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `is_convertible_v<decltype(u), T>`, `is_constructible_v<T, decltype(u)>`, and `is_same_v<T, U>` are all `false`.

21 *Effects:* For all *i*, initializes the i^{th} element of `*this` with `get<`*`i`*`>(`*`FWD`*`(u))`.

22 *Remarks:* The expression inside `explicit` is equivalent to: `!(is_convertible_v<decltype(get<I>(`*`FWD`*`(u))), Types> && ...)`. The constructor is defined as deleted if `(reference_constructs_from_temporary_v<Types, decltype(get<I>(`*`FWD`*`(u)))> || ...)` is `true`.

```
template<class U1, class U2> constexpr explicit(see below) tuple(pair<U1, U2>& u);
template<class U1, class U2> constexpr explicit(see below) tuple(const pair<U1, U2>& u);
template<class U1, class U2> constexpr explicit(see below) tuple(pair<U1, U2>&& u);
template<class U1, class U2> constexpr explicit(see below) tuple(const pair<U1, U2>&&
        u);
```

23 Let *`FWD`*`(u)` be `static_cast<decltype(u)>(u)`.

24 *Constraints*:

(24.1) — `sizeof...(Types)` is 2 and

(24.2) — `is_constructible_v<`T_0`, decltype(get<0>(`*`FWD`*`(u)))>` is `true` and

(24.3) — `is_constructible_v<`T_1`, decltype(get<1>(`*`FWD`*`(u)))>` is `true`.

25 *Effects:* Initializes the first element with `get<0>(`*`FWD`*`(u))` and the second element with `get<1>(`*`FWD`*`(u))`.

26 *Remarks:* The expression inside `explicit` is equivalent to: `!is_convertible_v<decltype(get<0>(FWD(u))), T0> || !is_convertible_v<decltype(get<1>(FWD(u))), T1>` The constructor is defined as deleted if `reference_constructs_from_temporary_v<T0, decltype(get<0>(FWD(u)))>` is `true` or `reference_constructs_from_temporary_v<T1, decltype(get<1>(FWD(u)))>` is `true`.

6. Edit 20.5.5 [tuple.apply] as indicated:

```
template<class T, class Tuple>
  constexpr T make_from_tuple(Tuple&& t);
```

? *Mandates:* If `tuple_size_v<remove_reference_t<Tuple>>` is `1`, then `reference_constructs_from_temporary_v<T, decltype(get<0>(declval<Tuple>()))>` is `false`.

2 *Effects:* Given the exposition-only function:

```
template<class T, class Tuple, size_t... I>
  requires is_constructible_v<T, decltype(get<I>(declval<Tuple>
        ()))...>
constexpr T make-from-tuple-impl(Tuple&& t, index_sequence<I...>) {
        // exposition only
  return T(get<I>(std::forward<Tuple>(t))...);
}
```

Equivalent to:

```
return make-from-tuple-impl<T>(
           std::forward<Tuple>(t),

        make_index_sequence<tuple_size_v<remove_reference_t<Tuple>>>
        {});
```

[*Note 1:* The type of `T` must be supplied as an explicit template parameter, as it cannot be deduced from the argument list. — *end note*]

7. Edit 20.14.4 [func.require] p2 as indicated:

2 Define *INVOKE*`<R>(f, t1, t2, ... , tN )` as `static_cast<void>(`*INVOKE*`(f, t1, t2, ... , tN ))` if `R` is *cv* `void`, otherwise *INVOKE*`(f, t1, t2, ... , tN )` implicitly converted to `R`. If `reference_converts_from_temporary_v<R, decltype(`*INVOKE*`(f, t1, t2, ... , tN))>` is `true`, *INVOKE*`<R>(f, t1, t2, ... , tN )` is ill-formed.

8. Add the following feature-test macro to 17.3.2 [version.syn], header `<version>` synopsis:

```
#define __cpp_lib_reference_from_temporary 20XXXXL // also in <type_traits>
```

§ 8 References

[CWG1696] Richard Smith. 2013-05-31. Temporary lifetime and non-static data member initializers. https://wg21.link/cwg1696

[gcc-pr95153] Alisdair Meredith. 2020. Bug 95153 - Arrays of `const void *` should not be copyable in C++20. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=95153

[LWG2813] Brian Bi. std::function should not return dangling references. https://wg21.link/lwg2813

[LWG3121] Matt Calabrese. tuple constructor constraints for UTypes&&… overloads.
https://wg21.link/lwg3121

[LWG3440] Ville Voutilainen. Aggregate-paren-init breaks direct-initializing a tuple or optional from {aggregate-member-value}.
https://wg21.link/lwg3440

[N4892] Thomas Köppe. 2021-06-18. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4892

[P0792R5] Vittorio Romeo. 2019-10-06. function_ref: a non-owning reference to a Callable.
https://wg21.link/p0792r5

[P0932R1] Aaryaman Sagar. 2018-02-07. Tightening the constraints on std::function.
https://wg21.link/p0932r1

[P2321R2] Tim Song. 2021-06-11. zip.
https://wg21.link/p2321r2